

PINNs vs ODE Solvers Numerical Experiments Benchmark

an Advanced Numerical Methods project

ANDREA SPINELLI¹

¹Computer Science Engineering Department, Università degli Studi di Palermo, Italy

September 10, 2025

Abstract

Questo lavoro presenta una valutazione comparativa tra solutori numerici classici per equazioni differenziali ordinarie e reti neurali informate dalla fisica (PINN). L'analisi considera problemi con complessità crescente e adotta criteri standard di accuratezza ed efficienza, includendo scenari con dati sintetici e reali. I risultati evidenziano come i metodi tradizionali rimangano un riferimento per stabilità e costi computazionali in contesti ben posti, mentre le PINN risultano competitive quando è necessario integrare vincoli fisici con dati scarsi o rumorosi e quando si ricercano modelli surrogati flessibili. Il contributo principale è un quadro sperimentale riproducibile che mette a confronto i due approcci in modo sistematico, fornendo indicazioni pratiche su quando privilegiare l'uno o l'altro in funzione degli obiettivi (accuratezza, generalizzazione guidata dai dati, costo computazionale).

Keywords: *Physics-Informed Neural Networks (PINN), equazioni differenziali ordinarie (ODE), solutori numerici, Euler esplicito, Euler implicito, Crank–Nicolson, Leapfrog, Runge–Kutta, oscillatore di Van der Pol, attrattore di Lorenz, sistemi stiff, dinamica caotica, benchmark sperimentale, metriche di accuratezza, modelli surrogati, integrazione dati-fisica, riproducibilità*

Contents

1	Introduction	4
1.1	Background and Motivations	4
1.2	Limitations of Traditional Methods	5
1.2.1	Mesh Dependence	5
1.2.2	The Curse of Dimensionality	5
1.2.3	Computational Rigidity and "Stiff" Systems	5
1.3	Main Objectives and Contributions	6
2	Background	7
2.1	Physical Systems and Mathematical Models	7
2.1.1	Hooke's Law: The Foundation of Elasticity	7
2.1.2	Simple Harmonic Motion: The Ideal Oscillator	7
2.1.3	Double Pendulum: Complexity in Chaos	8
2.1.4	Van der Pol Oscillator: Limit Cycles and Computational Rigidity	8
2.1.5	Jerk Oscillator: Higher Order Dynamics	9
2.1.6	Blasius Problem: From PDEs to ODEs in Fluid Dynamics	9
2.1.7	The Three-Body Problem: Chaos in the Cosmos	9
2.2	Numerical Methods for ODEs	10
2.2.1	Euler Explicit	10
2.2.2	Euler Implicit	10
2.2.3	Crank-Nicolson (Trapezoid Method)	10
2.2.4	Leapfrog	11
2.2.5	Runge-Kutta	11
2.3	Physics-Informed Neural Networks (PINN)	12
2.3.1	The Neural Network as a Solution Approximator	12
2.3.2	The Physically Informed Cost Function	12
2.3.3	Automatic Differentiation: The PINN Engine	13
2.3.4	Training Process	13
3	Methodology	14
3.1	Problem Test	14
3.1.1	Mathematical Formulation	14
3.1.2	Conversion to First Order System	14
3.2	Numerical Methods Implementations	15
3.2.1	Explicit Method Implementations (Euler Explicit, Leapfrog, Runge-Kutta)	15
3.2.2	Implicit Method Implementations (Euler Implicit, Crank-Nicolson)	15
3.3	PINN Implementation	18
3.3.1	Neural Network Architecture	18
3.3.2	Cost Function and Sampling Points	18
3.3.3	Optimization and Training	19
3.4	Evaluation Metrics	25
3.4.1	Mean Squared Error - MSE	25
3.4.2	Normalized Mean Squared Error - NMSE	25
3.4.3	Mean Absolute Error - MAE	25
3.4.4	Coefficient of Determination - R2	25
4	Results	26
4.1	Dataset and Preprocessing	26
4.2	Sperimental Setups	26
4.2.1	Hardware environment	26
4.2.2	Software environment	27

4.3	ODEs vs PINNs Benchmark	27
4.4	Code Availability	27
5	Conclusions	28
5.1	Results interpretation	28
5.2	Sperimental Implications	28
5.3	Strengths and Limitations	29
5.3.1	Strengths	29
5.3.2	Limits and Future Prospects	29
6	Bibliografia	30

1 Introduction

La risoluzione delle equazioni differenziali ordinarie (ODE) è fondamentale per la modellazione dei sistemi dinamici, un compito storicamente affidato a metodi numerici tradizionali. Tuttavia, l'efficienza di questi approcci consolidati è spesso minata da sfide quali la rigidità computazionale (stiffness), la dipendenza da griglie di calcolo e la "maledizione della dimensionalità". In risposta a tali limiti, le Physics-Informed Neural Networks (PINN) sono emerse come un potente paradigma alternativo, in grado di integrare le leggi fisiche direttamente in un processo di apprendimento "mesh-free". Questo capitolo introduttivo delinea il contesto di questo dualismo, analizzando le debolezze dei metodi classici per poi definire con precisione gli obiettivi e i contributi di questo studio, che culminerà in un confronto diretto tra le due metodologie.

1.1 Background and Motivations

Le equazioni differenziali ordinarie (ODE) costituiscono la spina dorsale della modellazione matematica in innumerevoli discipline scientifiche e ingegneristiche, dalla fisica alla biologia, dalla chimica all'epidemiologia [AOE25]. Esse descrivono l'evoluzione temporale dei sistemi dinamici, permettendo di prevederne il comportamento a partire da un insieme di condizioni iniziali. Data la complessità della maggior parte di questi sistemi, in particolare quelli non lineari, la ricerca di soluzioni analitiche in forma chiusa è spesso impraticabile, rendendo necessario il ricorso a metodi numerici.

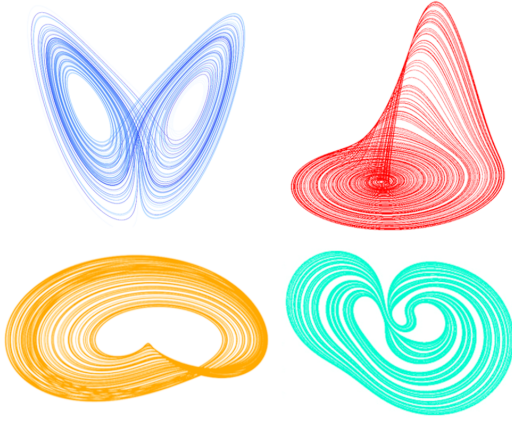


Fig.1.0 - Some Chaotic Dynamics

Per decenni, i metodi numerici classici, come quelli basati su differenze finite o le famiglie di metodi Runge-Kutta, hanno rappresentato lo standard de facto per l'approssimazione delle soluzioni di ODE [Cha+25]. La loro robustezza, accuratezza e le solide garanzie teoriche sulla convergenza li hanno resi strumenti indispensabili nel calcolo scientifico. Tuttavia, questi approcci "mesh-based" presentano alcune limitazioni intrinseche. La loro efficienza è strettamente legata alla discretizzazione del dominio spazio-temporale, un processo che può diventare computazionalmente proibitivo, specialmente per sistemi "stiff", i quali richiedono passi

di integrazione estremamente piccoli per mantenere la stabilità numerica [ND22]. Inoltre, l'uso di solutori numerici tradizionali risulta problematico in contesti di ottimizzazione, come nei problemi inversi per la stima di parametri o nel controllo ottimo, dove l'ODE deve essere risolta ripetutamente per ogni valutazione della funzione obiettivo [Cha+25]. In questo scenario, i recenti progressi nel campo dell'apprendimento automatico hanno aperto la strada a un nuovo paradigma: le Physics-Informed Neural Networks (PINN). Le PINN sono una classe di reti neurali profonde progettate per integrare le leggi fisiche, espresse sotto forma di equazioni differenziali, direttamente nel processo di apprendimento. A differenza dei modelli puramente data-driven, che necessiterebbero di una grande quantità di dati etichettati per apprendere la dinamica di un sistema, le PINN sfruttano la conoscenza a priori del modello fisico. L'architettura di una PINN approssima la soluzione dell'ODE, $u(t)$, tramite una rete neurale, $u_N(t, \theta)$. La funzione di costo (loss function) non viene minimizzata solo sui dati noti (es. condizioni iniziali), ma include anche un termine di regolarizzazione che penalizza la deviazione della rete dal soddisfare l'equazione differenziale stessa. Questo "residuo fisico" viene calcolato su un insieme di punti di collocazione distribuiti nel dominio, rendendo l'approccio intrinsecamente "mesh-free" [Ziy24].

Questo framework offre vantaggi significativi, tra cui la capacità di operare con dati sparsi o rumorosi ("small data regime") e di fornire una soluzione analitica approssimata, differenziabile e valida in tutto il dominio continuo [Lu+24]. Ciononostante, anche le PINN presentano delle sfide, in particolare nella gestione di sistemi fortemente non lineari o "stiff", dove l'ottimizzazione della funzione di costo può soffrire di flussi di gradiente sbilanciati ("gradient flow pathologies") [Bat23]. La ricerca è quindi attivamente impegnata nello sviluppo di architetture e strategie di addestramento più robuste. Il presente lavoro si inserisce in questo contesto dinamico, con l'obiettivo di confrontare le performance dei solutori

numerici tradizionali con quelle di una rete PINN nell'analisi di sistemi di ODE di interesse.

1.2 Limitations of Traditional Methods

Nonostante il loro consolidato successo, i metodi numerici tradizionali per la risoluzione di ODE presentano delle limitazioni fondamentali che ne restringono l'applicabilità e l'efficienza in scenari specifici. Queste sfide, che vanno dalla gestione di geometrie complesse alla risoluzione di problemi mal condizionati, hanno catalizzato la ricerca di approcci alternativi, tra cui le Physics-Informed Neural Networks. Di seguito, analizziamo i tre limiti principali di interesse per questo lavoro.

1.2.1 Mesh Dependence

Il fondamento dei metodi numerici classici, come le differenze finite, gli elementi finiti o i metodi Runge-Kutta, risiede nella discretizzazione del dominio continuo in un insieme finito di punti o elementi, noto come griglia computazionale (o mesh). La soluzione viene calcolata in questi punti discreti e poi, se necessario, interpolata per ottenere un'approssimazione continua. Sebbene questo approccio sia efficace, introduce un forte accoppiamento tra l'accuratezza della soluzione e la densità della griglia. Per catturare dinamiche complesse o gradienti elevati, è necessario infittire la griglia, aumentando drasticamente il numero di punti e, di conseguenza, il costo computazionale e la memoria richiesta. Questo trade-off è un vincolo intrinseco: la complessità di un solutore basato su griglia spesso scala polinomialmente con la riduzione del passo di integrazione h , limitando l'efficienza in problemi che richiedono alta precisione [Ziy24].

1.2.2 The Curse of Dimensionality

Una delle sfide più significative del calcolo scientifico è la cosiddetta "maledizione della dimensionalità". Questo fenomeno descrive la crescita esponenziale del volume computazionale richiesta per analizzare un problema all'aumentare del numero delle sue dimensioni. Nei metodi basati su griglia, questo si manifesta in modo lampante: se per discretizzare una singola dimensione sono necessari N punti, per un problema in D dimensioni una griglia regolare ne richiederà N^D . Questo rende rapidamente intrattabile l'analisi di sistemi in alta dimensione. Sebbene le ODE siano tipicamente problemi monodimensionali nel tempo, la maledizione della dimensionalità

emerge prepotentemente in contesti parametrici. Ad esempio, quando si desidera ottenere un "modello surrogato" che descriva la soluzione di un'ODE non per un singolo set di parametri, ma per un intero range di parametri iniziali $\theta \in \Theta$ e condizioni iniziali $x_0 \in \Omega$, il problema diventa intrinsecamente multi-dimensionale [Cha+25]. Risolvere tali problemi con metodi classici richiederebbe di eseguire un'enorme quantità di simulazioni separate per campionare lo spazio dei parametri, un approccio noto come "brute-force" che scala molto male. Le PINN, essendo "mesh-free", offrono una via promettente per superare questa barriera [Ziy24; Lu+24].

1.2.3 Computational Rigidity and "Stiff" Systems

Un'altra classe di problemi notoriamente ostica per i solutori numerici è quella dei sistemi "stiff". Un'ODE è definita stiff quando la sua soluzione contiene componenti che evolvono su scale temporali molto diverse. In altre parole, coesistono dinamiche "veloci" che si esauriscono rapidamente e dinamiche "lente" che determinano l'evoluzione a lungo termine del sistema. La "stiffness" pone una seria sfida ai metodi espliciti (come i metodi ODE nella funzione `ode45` di MATLAB). Per garantire la stabilità numerica, il passo di integrazione h di un metodo esplicito deve essere sufficientemente piccolo da risolvere la scala temporale più veloce del sistema, anche se tale dinamica è transitoria e di scarso interesse per l'analisi complessiva [Bat23; Gal+21]. Formalmente, per un'equazione test $y' = \lambda y$ con $\operatorname{Re}(\lambda) \ll 0$, la stabilità richiede che h sia dell'ordine di $1/|\lambda|$. Se $|\lambda|$ è molto grande, h diventa proibitivamente piccolo. Questo vincolo sulla stabilità, e non sull'accuratezza, rende i metodi espliciti estremamente inefficienti per i sistemi stiff [ND22].

Per ovviare a questo problema, sono stati sviluppati metodi impliciti (come le Formule di Differenziazione a ritroso, BDF), che possiedono regioni di stabilità molto più ampie. Tuttavia, questa stabilità ha un costo: ad ogni passo temporale, un metodo implicito richiede la soluzione di un sistema di equazioni, spesso non lineari, un'operazione computazionalmente più onerosa rispetto a una valutazione esplicita [Gal+21]. Le PINN, operando in modo fondamentalmente diverso—ottimizzando una rappresentazione globale della soluzione—offrono un approccio alternativo per affrontare la "stiffness" senza dipendere da un passo di integrazione vincolato dalla stabilità locale [Flo+24].

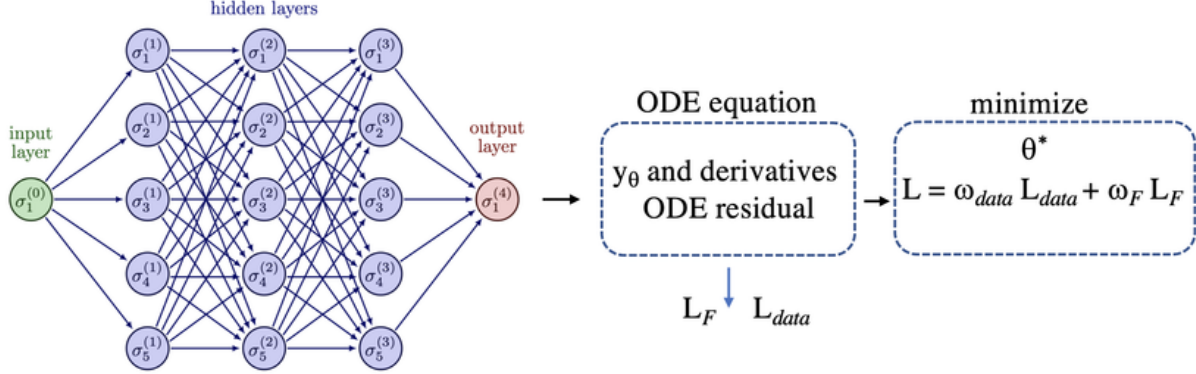


Fig.1.1 - Physic-Informed Neural Network (PINN)

1.3 Main Objectives and Contributions

Alla luce delle limitazioni intrinseche dei metodi numerici classici e delle promettenti, sebbene ancora in fase di sviluppo, capacità delle Physics-Informed Neural Networks, emerge la necessità di una valutazione rigorosa e comparativa delle due metodologie. Comprendere dove e perché un approccio possa essere preferibile all'altro è cruciale per guidare la scelta dello strumento più adatto a seconda del problema in esame.

Il presente lavoro si inserisce in questo dibattito con un obiettivo chiaro e mirato: confrontare sistematicamente le performance di una selezione di metodi numerici tradizionali per ODE con quelle di una rete PINN nella risoluzione di un sistema differenziale che esibisce un comportamento complesso e potenzialmente caotico. Per raggiungere tale obiettivo, questo studio si propone di rispondere alle seguenti domande di ricerca:

- **Accuratezza vs. Efficienza:** A parità di accuratezza richiesta, qual è l'efficienza computazionale dei diversi metodi? E, viceversa, a parità di tempo di calcolo, quale approccio garantisce la soluzione più accurata?
- **Gestione della Complessità:** Come si comportano i diversi solutori di fronte a dinamiche non lineari e "stiff", dove i metodi classici possono incontrare difficoltà di stabilità o richiedere costi computazionali elevati?
- **Generalizzazione e Flessibilità:** Una PINN, una volta addestrata, può agire come un modello surrogato efficiente, fornendo soluzioni istantanee per diverse condizioni iniziali, superando così la necessità di rieseguire da capo una simulazione numerica?

Per rispondere a queste domande, questo paper offre i seguenti contributi principali:

1. **Benchmark Quantitativo:** Si fornisce una valutazione numerica dettagliata, confrontando una serie di solutori classici — tra cui Eulero Esplicito, Eulero Implicito, Crank-Nicolson e il metodo Runge-Kutta di quarto ordine (RK4) — con un'architettura PINN. Le performance vengono misurate in termini di:
 - **Accuratezza**, utilizzando metriche standard come l'Errore Assoluto Medio (MAE) e l'Errore Quadratico Medio (MSE) rispetto a una soluzione di riferimento ad alta precisione.
 - **Efficienza Computazionale**, misurando il tempo di esecuzione (wall-clock time) richiesto da ciascun metodo per raggiungere una data tolleranza.
2. **Analisi Critica in un Contesto Complesso:** L'analisi è focalizzata su un sistema di ODE noto per la sua complessità, permettendo di evidenziare i punti di forza e di debolezza di ciascun approccio in condizioni non banali, come la gestione dell'accumulo di errore su lunghi orizzonti temporali e la cattura di traiettorie caotiche.
3. **Implementazione Open-Source:** A supporto della trasparenza e della riproducibilità della ricerca, viene rilasciato il codice sorgente completo utilizzato per gli esperimenti, comprendente l'implementazione di tutti i solutori numerici e dell'architettura PINN. Questo codice funge da base pratica per ulteriori studi e confronti.

2 Background

Questo capitolo pone le fondamenta teoriche necessarie per l'analisi comparativa. Verranno dapprima introdotti i modelli matematici dei sistemi fisici utilizzati come banchi di prova, che spaziano da oscillatori lineari a dinamiche caotiche. Successivamente, si esaminerà la formulazione teorica dei metodi numerici classici, per poi introdurre in dettaglio il paradigma delle Physics-Informed Neural Networks (PINN). L'obiettivo è fornire al lettore gli strumenti concettuali per comprendere sia i problemi analizzati sia i principi dei due approcci metodologici a confronto.

2.1 Physical Systems and Mathematical Models

Le equazioni differenziali non nascono in un vuoto teorico, ma sono il linguaggio matematico utilizzato per descrivere l'evoluzione dei sistemi fisici. Comprendere l'origine fisica di un'equazione è fondamentale per interpretarne le soluzioni e valutarne l'accuratezza. Di seguito, vengono presentati i modelli teorici dei sistemi scelti per questo studio, che spaziano da semplici oscillatori lineari a complessi problemi caotici.

2.1.1 Hooke's Law: The Foundation of Elasticity

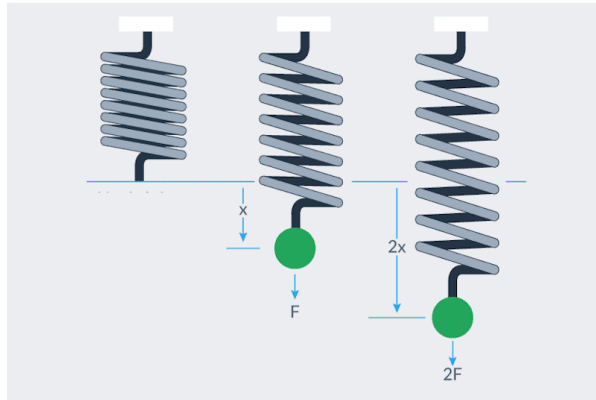


Fig.2.0 - Hooke's Law

La legge di Hooke è un principio fondamentale della meccanica che descrive il comportamento dei materiali elastici. Nella sua forma più semplice, applicata a una molla ideale, afferma che la forza (F) necessaria per deformare (allungare o comprimere) la molla di una certa distanza (x) dalla sua posizione di equilibrio è direttamente proporzionale a quella distanza. Matematicamente, è espressa come:

$$F = -kx$$

dove:

- F è la **forza di richiamo elastica** esercitata dalla molla.

- k è la **costante elastica** della molla (o *stiffness*), che ne misura la rigidità. Un valore di k elevato indica una molla molto rigida.
- x è lo **spostamento** dalla posizione di equilibrio.

Il segno negativo indica che la forza elastica si oppone sempre alla direzione dello spostamento, tendendo a riportare il sistema alla sua posizione di equilibrio. Questa legge, pur essendo un'idealizzazione, è il punto di partenza per la descrizione di qualsiasi sistema oscillante.

2.1.2 Simple Harmonic Motion: The Ideal Oscillator

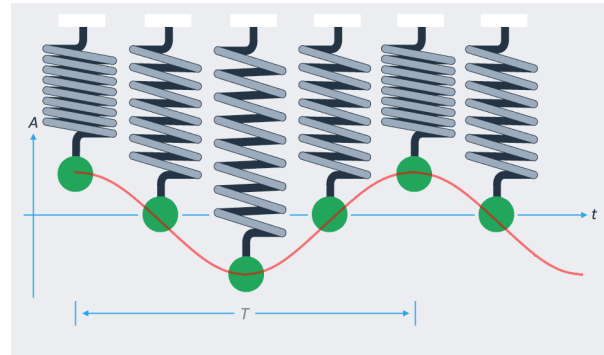


Fig.2.1 - Simple Harmonic Motion

Il moto armonico semplice è il modello di oscillazione più elementare e si ottiene applicando la seconda legge di Newton a una massa (m) soggetta unicamente alla forza di richiamo di una molla ideale (descritta dalla legge di Hooke).

Partendo dalla seconda legge di Newton:

$$F = ma = m \frac{d^2x}{dt^2} = m\ddot{x}$$

Eguagliando questa espressione alla forza di Hooke, si ottiene:

$$m\ddot{x} = -kx$$

Riorganizzando i termini, si arriva all'equazione differenziale ordinaria del secondo ordine, lineare e omogenea, che governa il moto armonico semplice:

$$m\ddot{x} + kx = 0$$

Dividendo per la massa m e definendo la **pulsazione angolare** $\omega_0 = \sqrt{k/m}$, l'equazione assume la sua forma canonica:

$$\ddot{x} + \omega_0^2 x = 0$$

Questo sistema è **conservativo**: l'energia totale, somma di energia cinetica e potenziale elastica, rimane costante nel tempo.

2.1.3 Double Pendulum: Complexity in Chaos

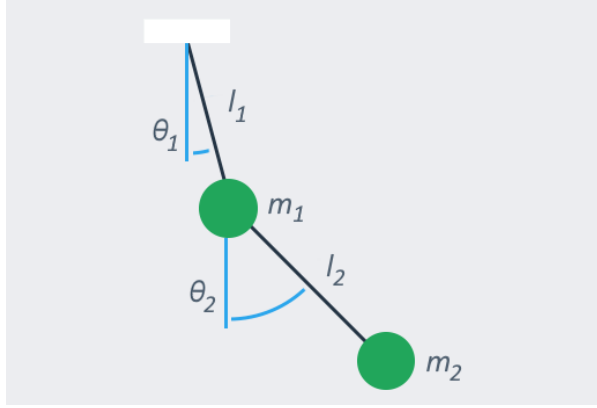


Fig.2.2 - Double Pendulum

Il doppio pendolo è uno dei più celebri esempi di come un sistema fisico apparentemente semplice possa esibire un comportamento caotico. È composto da un pendolo semplice alla cui massa è appeso un secondo pendolo. A differenza del pendolo singolo, il suo moto non è descrivibile con semplici funzioni armoniche.

Le equazioni del moto sono tipicamente derivate tramite la meccanica Lagrangiana, definendo le coordinate generalizzate come gli angoli θ_1 e θ_2 dei due pendoli rispetto alla verticale. Le equazioni risultanti sono un sistema di due ODE del secondo ordine, non lineari e accoppiate:

Per il primo pendolo:

$$f = (m_1 + m_2)l_1\ddot{\theta}_1 + m_2l_2\ddot{\theta}_2 \cos(\theta_1 - \theta_2) + m_2l_2\dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + (m_1 + m_2)g \sin(\theta_1) = 0 \quad (1)$$

Per il secondo pendolo:

$$m_2l_2\ddot{\theta}_2 + m_2l_1\dot{\theta}_1 \cos(\theta_1 - \theta_2) - m_2l_1\dot{\theta}_1^2 \sin(\theta_1 - \theta_2) + m_2g \sin(\theta_2) = 0 \quad (2)$$

dove m_1, m_2 sono le masse, l_1, l_2 le lunghezze dei pendoli e g l'accelerazione di gravità. Il sistema è fortemente **non lineare** a causa dei termini trigonometrici e dei prodotti tra le variabili e le loro derivate. È un sistema **conservativo** (in assenza di attrito) ma la sua evoluzione è estremamente **sensibile alle condizioni iniziali**, una caratteristica distintiva del caos deterministico.

2.1.4 Van der Pol Oscillator: Limit Cycles and Computational Rigidity

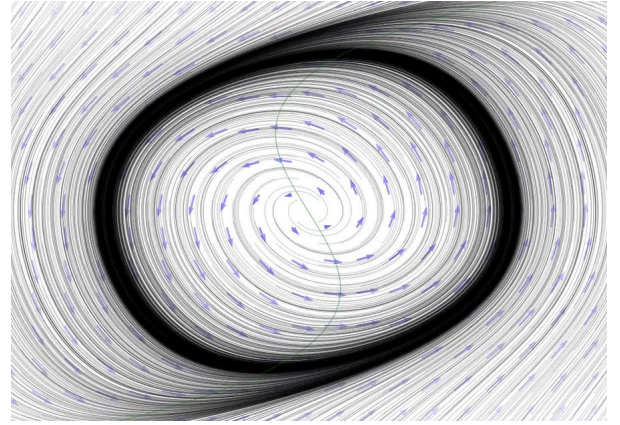


Fig.2.3 - Van der Pol Oscillator

L'oscillatore di Van der Pol è un sistema dinamico non conservativo, storicamente sviluppato per modellare i circuiti elettronici a valvole. La sua caratteristica più notevole è la presenza di un **ciclo limite** indipendentemente dalle condizioni iniziali (eccetto l'origine), la traiettoria del sistema tende a un'unica orbita periodica stabile nello spazio delle fasi.

L'equazione che lo descrive è una modifica dell'oscillatore armonico, con l'aggiunta di un termine di smorzamento non lineare:

$$\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$$

Il parametro $\mu \geq 0$ controlla la non linearità e lo smorzamento del sistema:

- Se $\mu = 0$, l'equazione si riduce a quella del moto armonico semplice.
- Se $\mu > 0$, il termine di smorzamento è **negativo** per $|x| < 1$ (introduce energia, allontanando il sistema dall'origine) e **positivo** per $|x| > 1$ (dissipa energia, riportando il sistema verso l'origine).

Per valori elevati di μ , il sistema diventa estremamente **stiff** con periodi di lenta accumulazione di energia seguiti da rapidissimi rilasci. Questo lo rende un banco di prova classico e impegnativo per i solutori numerici.

2.1.5 Jerk Oscillator: Higher Order Dynamics

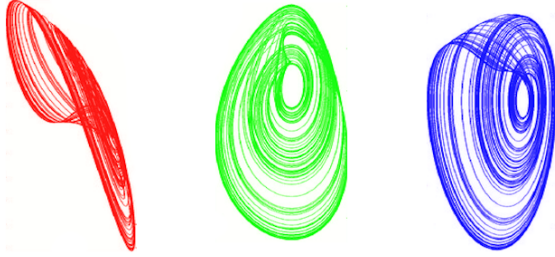


Fig.2.4 - Jerk Oscillator

Un oscillatore Jerk è un sistema la cui dinamica è descritta da un'equazione differenziale del terzo ordine. Il nome deriva dal termine "jerk" (in italiano "strappo"), che in fisica indica la derivata terza della posizione rispetto al tempo, ovvero la variazione dell'accelerazione.

La forma generale di un'equazione di jerk è:

$$\ddot{x} = f(x, \dot{x}, \ddot{x})$$

Queste equazioni sono le più semplici, in termini di ordine differenziale, in grado di esibire un comportamento caotico. Un esempio noto è l'oscillatore di Sprott, definito da:

$$\ddot{x} + A\ddot{x} - \dot{x}^2 + x = 0$$

A seconda del valore del parametro A , questo sistema può mostrare un comportamento periodico, quasi-periodico o caotico, rappresentando una sfida numerica legata al suo ordine superiore.

2.1.6 Blasius Problem: From PDEs to ODEs in Fluid Dynamics

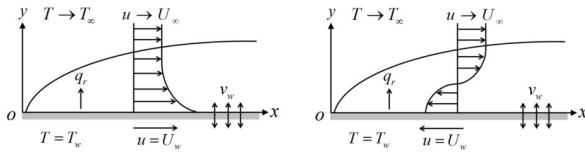


Fig.2.5 - Blasius Problem

Il problema di Blasius è un classico della fluidodinamica che descrive il profilo di velocità di un flusso laminare stazionario che si sviluppa sopra una

lastra piana semi-infinita (strato limite). Sebbene il problema fisico sia governato dalle equazioni di Navier-Stokes (un sistema di PDE), attraverso una trasformazione di similarità è possibile ridurlo a una singola ODE non lineare del terzo ordine.

L'equazione di Blasius è data da:

$$2f'''(\eta) + f(\eta)f''(\eta) = 0$$

dove f è una funzione adimensionale legata alla funzione di corrente del flusso e η è la variabile di similarità che combina le coordinate spaziali. A differenza dei precedenti, questo è un **problema ai valori al contorno (BVP)** su un dominio semi-infinito, con le condizioni:

$$f(0) = 0, \quad f'(0) = 0, \quad \lim_{\eta \rightarrow \infty} f'(\eta) = 1$$

La non linearità e le condizioni al contorno su un dominio infinito lo rendono un interessante problema test.

2.1.7 The Three-Body Problem: Chaos in the Cosmos

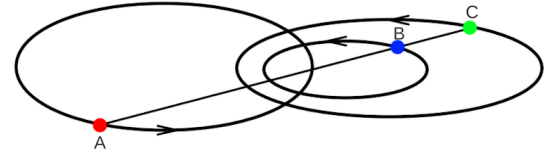


Fig.2.6 - The Three-Body Problem

Il problema dei tre corpi è uno dei problemi più antichi e famosi della meccanica celeste. Esso consiste nel determinare il moto di tre corpi puntiformi che interagiscono tra loro unicamente attraverso la reciproca attrazione gravitazionale.

Applicando la legge di gravitazione universale di Newton e la sua seconda legge del moto, si ottiene un sistema di equazioni differenziali vettoriali del secondo ordine. Per ciascuna massa m_i con vettore posizione $\mathbf{r}_i$, l'equazione è:

$$\ddot{\mathbf{r}}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{\|\mathbf{r}_j - \mathbf{r}_i\|^3}$$

dove G è la costante di gravitazione universale. Questo sistema descrive il moto dei tre corpi nello spazio tridimensionale. Scritto in coordinate cartesiane, si traduce in un sistema di 9 equazioni differenziali del secondo ordine (una per ogni coordinata di ogni corpo), che a sua volta può essere convertito in un sistema di **18 equazioni del primo ordine**.

2.2 Numerical Methods for ODEs

Quando una soluzione analitica a un'equazione differenziale non è disponibile, si ricorre a metodi numerici. L'idea fondamentale alla base di questi metodi è la **discretizzazione** il dominio continuo del tempo viene suddiviso in una serie di istanti discreti $t_0, t_1, t_2, \dots, t_N$, separati da un passo di integrazione $h = t_{n+1} - t_n$. A partire da una condizione iniziale nota $y(t_0) = y_0$, il solutore approssima la soluzione y_{n+1} al tempo t_{n+1} basandosi sulle informazioni disponibili al tempo t_n (e talvolta ai tempi precedenti). La qualità di un solutore è determinata dalla sua accuratezza, stabilità e costo computazionale.

Consideriamo il problema generico ai valori iniziali (IVP):

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0$$

2.2.1 Euler Explicit

È il metodo numerico più semplice e intuitivo. L'idea è di approssimare la soluzione al passo successivo utilizzando una progressione lineare basata sulla pendenza della funzione al punto attuale.

Partiamo dall'espansione in serie di Taylor della soluzione esatta $y(t)$ attorno al punto t_n :

$$y(t_{n+1}) = y(t_n + h) = y(t_n) + hy'(t_n) + \frac{h^2}{2!}y''(\xi)$$

per un qualche $\xi \in (t_n, t_{n+1})$. Sappiamo dall'equazione differenziale che $y'(t_n) = f(t_n, y(t_n))$. Sostituendo questa espressione e troncando la serie al primo ordine, commettiamo un **errore di troncamento locale** (LTE) di ordine $O(h^2)$. Sostituendo l'approssimazione numerica y_n al posto della soluzione esatta $y(t_n)$, otteniamo la formula del metodo.

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Il metodo di Eulero Esplicito è **esplicito** perché y_{n+1} è calcolato direttamente da valori noti. È computazionalmente molto economico e semplice da implementare. Tuttavia, la sua accuratezza è bassa, con un **errore di troncamento globale** (l'errore accumulato dopo molti passi) di ordine $O(h)$. La sua principale debolezza è la scarsa **stabilità** è utilizzabile solo con passi di integrazione h molto piccoli, specialmente per sistemi **stiff** rendendolo inadatto a molti problemi pratici.

2.2.2 Euler Implicit

Questo metodo è la controparte implicita del precedente. Invece di usare la pendenza nel punto corrente t_n , utilizza la pendenza nel punto **futuro** e sconosciuto t_{n+1} .

In questo caso, l'espansione di Taylor viene eseguita all'indietro, dal punto t_{n+1} :

$$y(t_n) = y(t_{n+1} - h) = y(t_{n+1}) - hy'(t_{n+1}) + \frac{h^2}{2!}y''(\xi)$$

Risolvendo per $y(t_{n+1})$ e sapendo che $y'(t_{n+1}) = f(t_{n+1}, y(t_{n+1}))$, otteniamo:

$$y(t_{n+1}) = y(t_n) + hf(t_{n+1}, y(t_{n+1})) + O(h^2)$$

Troncando il termine di errore, si ottiene la formula.

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Il metodo è **implicito** poiché l'incognita y_{n+1} appare su entrambi i lati dell'equazione. Per risolvere questa equazione ad ogni passo temporale, è necessario utilizzare un metodo di ricerca della radice (come il metodo di Newton), rendendolo computazionalmente più costoso del suo omologo esplicito. Il suo grande vantaggio è l'**eccellente stabilità**. È un metodo **A-stabile** il che significa che è stabile per qualsiasi passo h per sistemi stiff, rendendolo una scelta robusta per questo tipo di problemi, nonostante la sua accuratezza sia ancora del primo ordine, $O(h)$.

2.2.3 Crank-Nicolson (Trapezoid Method)

Questo metodo combina i due approcci precedenti, calcolando la pendenza come la media tra quella nel punto corrente e quella nel punto successivo.

Partiamo dalla forma integrale dell'ODE:

$$\int_{t_n}^{t_{n+1}} y'(t)dt = y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t))dt$$

Approssimiamo l'integrale a destra utilizzando la **regola del trapezio**

$$\int_a^b g(t)dt \approx \frac{b-a}{2}(g(a) + g(b))$$

Applicando questa regola con $a = t_n$ e $b = t_{n+1}$, otteniamo la formula del metodo.

$$y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$

Il metodo di Crank-Nicolson è **implicito** e, come Eulero Implicito, richiede la soluzione di un'equazione ad ogni passo. Tuttavia, il suo **errore di troncamento globale** è di ordine $O(h^2)$, un significativo miglioramento in termini di accuratezza. È anch'esso **A-stabile** combinando i vantaggi di stabilità dei metodi impliciti con un'accuratezza superiore, rendendolo una scelta molto popolare e bilanciata.

2.2.4 Leapfrog

Il metodo Leapfrog è un metodo **esplicito a due passi**. A differenza dei metodi precedenti, per calcolare y_{n+1} non usa solo le informazioni in t_n , ma anche quelle in t_{n-1} .

L'idea è di approssimare la derivata al tempo t_n usando una **differenza finita centrata** che è più accurata della differenza in avanti (usata da Eulero Esplicito):

$$y'(t_n) \approx \frac{y(t_{n+1}) - y(t_{n-1}))}{2h}$$

Sostituendo $y'(t_n) = f(t_n, y_n)$ e risolvendo per y_{n+1} , si ottiene la formula.

$$y_{n+1} = y_{n-1} + 2hf(t_n, y_n)$$

Il metodo è **esplicito** quindi computazionalmente economico. Ha un'accuratezza del secondo ordine, $O(h^2)$. Il suo principale vantaggio è che è un **integratore simplettico** il che significa che conserva eccellentemente l'energia in sistemi Hamiltoniani (conservativi) su lunghi periodi di integrazione. Tuttavia, ha due svantaggi:

1. **Non è auto-avviante:** Per calcolare y_1 necessita di y_0 e y_{-1} (o, più comunemente, per calcolare y_2 necessita di y_1 e y_0). Il valore y_1 deve essere calcolato con un altro metodo (es. Eulero).
2. Può soffrire di un'instabilità debole in cui le soluzioni ai passi pari e dispari si disaccoppiano.

2.2.5 Runge-Kutta

I metodi Runge-Kutta (RK) sono una vasta e potente famiglia di metodi espliciti a un passo. L'idea è

di ottenere un'accuratezza di ordine superiore senza dover calcolare esplicitamente le derivate di ordine elevato di $y(t)$, come richiederebbe un'espansione di Taylor diretta. Questo si ottiene valutando la funzione $f(t, y)$ in diversi punti intermedi all'interno dell'intervallo $[t_n, t_{n+1}]$.

L'obiettivo è costruire un metodo della forma $y_{n+1} = y_n + h\Phi$ dove Φ è una media pesata di pendenze k_i . Per il metodo RK di quarto ordine (RK4), si usano quattro valutazioni. I pesi e i punti di valutazione sono scelti in modo tale che l'espansione di Taylor di y_{n+1} coincida con l'espansione di Taylor della soluzione esatta fino al termine di ordine h^4 . La derivazione completa è algebricamente complessa, ma garantisce che l'errore di troncamento locale sia $O(h^5)$, portando a un errore globale di $O(h^4)$.

Dato y_n al tempo t_n , si calcolano quattro pendenze intermedie:

$$\begin{aligned} k_1 &= f(t_n, y_n) \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) \\ k_4 &= f(t_n + h, y_n + hk_3) \end{aligned}$$

Il valore al passo successivo è quindi la media pesata di queste pendenze:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

RK4 è uno dei metodi più utilizzati per problemi non stiff. È **auto-avviante** ha un **eccellente accuratezza** ($O(h^4)$) ed è relativamente semplice da implementare. Sebbene richieda quattro valutazioni della funzione f per ogni passo, il guadagno in accuratezza compensa ampiamente il costo computazionale aggiuntivo rispetto a metodi di ordine inferiore. Come la maggior parte dei metodi espliciti, non è adatto per sistemi stiff. Esistono anche versioni implicite dei metodi Runge-Kutta che possiedono eccellenti proprietà di stabilità ma sono molto più complesse da implementare.

2.3 Physics-Informed Neural Networks (PINN)

Le Physics-Informed Neural Networks (PINN) rappresentano un'ibridazione tra il mondo dell'apprendimento profondo e i principi primi della fisica. L'idea fondamentale è sfruttare la straordinaria capacità delle reti neurali di agire come **approssimatori universali di funzioni** non per un mero fitting di dati, ma per apprendere una funzione che sia **soluzione di una specifica equazione differenziale**. Invece di trattare il sistema fisico come una "scatola nera", le PINN incorporano la conoscenza del modello matematico che lo governa direttamente nel processo di addestramento.

2.3.1 The Neural Network as a Solution Approximator

Il primo pilastro di una PINN è una rete neurale standard, tipicamente una rete **feedforward** (o Multi-Layer Perceptron, MLP). Questa rete viene utilizzata per creare una rappresentazione parametrica della soluzione incognita $y(t)$ dell'ODE. Chiamiamo questa approssimazione $y_{NN}(t, \theta)$.

- **Input:** La variabile indipendente, in questo caso il tempo t .
- **Output:** Il valore approssimato della soluzione y_{NN} al tempo t .
- **Parametri:** L'insieme di tutti i pesi (W) e bias (b) della rete, che denotiamo collettivamente con θ .

Matematicamente, per una rete con L strati nascosti, l'output può essere scritto come una composizione di funzioni:

$$y_{NN}(t, \theta) = \mathcal{L}_L \circ \sigma \circ \mathcal{L}_{L-1} \circ \dots \circ \sigma \circ \mathcal{L}_0(t)$$

dove:

- $\mathcal{L}_i(\mathbf{z}) = W_i \mathbf{z} + b_i$ è una trasformazione affine per lo strato i .
- $\sigma(\cdot)$ è una funzione di attivazione non lineare (es. 'tanh', 'sigmoid', 'ReLU') applicata elemento per elemento.
- $\theta = \{W_0, b_0, \dots, W_L, b_L\}$ è l'insieme completo dei parametri da ottimizzare.

È cruciale notare che, essendo le funzioni di attivazione tipicamente funzioni analitiche, $y_{NN}(t, \theta)$ è una funzione continua e differenziabile rispetto al

suo input t e ai suoi parametri θ . Questa proprietà è la chiave che permette di "informare" la rete con la fisica del problema.

2.3.2 The Physically Informed Cost Function

Il secondo e più innovativo pilastro delle PINN è la sua **funzione di costo (loss function)**. A differenza dell'addestramento supervisionato standard, dove l'errore è calcolato solo sui dati noti, la loss di una PINN è una combinazione di due (o più) termini:

1. **L'errore sui dati (Data Loss):** termine si assicura che la soluzione approssimata rispetti le condizioni note del problema (iniziali o al contorno).
2. **L'errore sulla fisica (Physics Loss):** termine forza la soluzione ad obbedire all'equazione differenziale stessa in tutto il suo dominio.

Consideriamo ancora il problema ai valori iniziali $\frac{dy}{dt} = f(t, y)$ con $y(t_0) = y_0$. La funzione di costo totale $L(\theta)$ è definita come:

$$L(\theta) = L_{data}(\theta) + \lambda L_{physics}(\theta)$$

dove λ è un iperparametro che bilancia l'importanza relativa dei due termini.

- **Derivazione della Data Loss (L_{data}):** Questo termine è tipicamente l'errore quadratico medio (MSE) calcolato sull'insieme dei dati noti. Per un IVP, questo insieme contiene solitamente solo la condizione iniziale $\{(t_0, y_0)\}$.

$$L_{data}(\theta) = |y_{NN}(t_0, \theta) - y_0|^2$$

Se fossero disponibili più punti dato $\{(t_i, y_i)\}_{i=1}^{N_d}$, la formula si generalizzerebbe a:

$$L_{data}(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} |y_{NN}(t_i, \theta) - y_i|^2$$

- **Derivazione della Physics Loss ($L_{physics}$):** Questo termine è il cuore della PINN. Definiamo il **residuo** dell'equazione differenziale come la quantità che *dovrebbe* essere zero se la soluzione fosse esatta: $r(t, \theta) = \frac{dy_{NN}(t, \theta)}{dt} - f(t, y_{NN}(t, \theta))$. L'obiettivo è minimizzare questo residuo non solo in un punto, ma su tutto il dominio di integrazione. Per farlo, si campiona un grande numero N_c di punti, detti **punti di collocazione** $\{t_{c,j}\}_{j=1}^{N_c}$, all'interno del dominio. La *physics loss*

è quindi l'errore quadratico medio del residuo su questi punti:

$$L_{physics}(\theta) = \frac{1}{N_c} \sum_{j=1}^{N_c} |r(t_{c,j}, \theta)|^2 = \frac{1}{N_c} \sum_{j=1}^{N_c} \left| \frac{dy_{NN}(t_{c,j}, \theta)}{dt} - f(t_{c,j}, y_{NN}(t_{c,j}, \theta)) \right|^2$$

I punti di collocazione sono "dati non supervisionati": non conosciamo il valore della soluzione in quei punti, ma conosciamo la legge fisica che deve soddisfare.

2.3.3 Automatic Differentiation: The PINN Engine

Una domanda sorge spontanea: come si calcola la derivata $\frac{dy_{NN}}{dt}$ necessaria per il residuo? I metodi tradizionali presentano svantaggi:

- La **differenziazione simbolica** è computazionalmente intrattabile per reti neurali complesse.
- La **differenziazione numerica** (es. differenze finite) introduce errori di troncamento che minerebbero l'accuratezza del metodo.

La soluzione è la **differenziazione automatica (AD)** una tecnica fondamentale nei moderni framework di deep learning (come TensorFlow e PyTorch). L'AD calcola le derivate esatte (a precisione di macchina) di una funzione espressa come un programma per computer. Poiché la rete neurale è una lunga sequenza di operazioni elementari (somme, prodotti, funzioni di attivazione), l'AD applica la **regola della catena (chain rule)** in modo sistematico attraverso il grafo computazionale della rete per ottenere la derivata dell'output rispetto a qualsiasi input o parametro.

Grazie all'AD, il calcolo del residuo $r(t, \theta)$ è sia efficiente che esatto, senza introdurre errori numerici.

2.3.4 Training Process

Il processo di addestramento di una PINN trasforma il problema di risolvere un'ODE in un problema di ottimizzazione. L'obiettivo è trovare l'insieme di parametri θ^* che minimizza la funzione di costo totale:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

Questo viene realizzato tramite un algoritmo di ottimizzazione basato su gradiente, come Adam o L-BFGS. I passi sono i seguenti:

1. **Inizializzazione:** I pesi e i bias θ della rete vengono inizializzati (solitamente con valori piccoli e casuali).
2. **Campionamento:** Si definisce l'insieme di punti dato (es. t_0) e si campionano i punti di collocazione $\{t_{c,j}\}$ dal dominio.
3. **Loop di Ottimizzazione:**
 - **Forward Pass:** Per un batch di punti dato e di collocazione, la rete calcola gli output y_{NN} e il residuo $r(t, \theta)$ (usando l'AD per la derivata).
 - **Calcolo della Loss:** Si calcolano L_{data} e $L_{physics}$ e si combinano per ottenere la loss totale $L(\theta)$.
 - **Backward Pass (Backpropagation):** L'AD viene usata una seconda volta per calcolare il gradiente della loss totale rispetto a ogni parametro della rete, $\nabla_{\theta} L(\theta)$.
 - **Aggiornamento dei Parametri:** L'ottimizzatore aggiorna i parametri θ nella direzione opposta a quella del gradiente per minimizzare la loss.
4. **Convergenza:** loop continua fino a quando la loss scende al di sotto di una soglia di tolleranza o si raggiunge un numero massimo di iterazioni.

Il risultato finale è una rete neurale $y_{NN}(t, \theta^*)$ che non solo rispetta le condizioni al contorno, ma la cui forma funzionale soddisfa l'equazione differenziale in tutto il dominio.

3 Methodology

Questo capitolo descrive in dettaglio l'impostazione e gli strumenti utilizzati per il confronto tra i solutori numerici classici e l'approccio basato su PINN. La prima sezione è dedicata alla definizione matematica del problema test scelto come banco di prova. Successivamente, verranno illustrate le specifiche di implementazione di ciascun metodo, le metriche di valutazione adottate per misurare le performance e l'architettura della rete neurale impiegata.

3.1 Problem Test

La scelta di un adeguato problema test è fondamentale per una valutazione equa e significativa dei diversi metodi numerici. Un buon banco di prova deve essere sufficientemente complesso da mettere in luce i punti di forza e di debolezza di ciascun approccio. Per la discussione dello studio, è stato selezionato l'**oscillatore di Van der Pol** un sistema dinamico non lineare che rappresenta un archetipo dei cicli limite e dei sistemi *stiff* (per l'analisi si sono testati tutti i problemi precedentemente descritti). Come si evince dal codice implementato nel repository del progetto, questo oscillatore offre la possibilità di modulare la sua "difficoltà" numerica semplicemente variando un singolo parametro, rendendolo ideale per un'analisi comparativa.

3.1.1 Mathematical Formulation

L'oscillatore di Van der Pol è descritto dalla seguente equazione differenziale ordinaria del secondo ordine:

$$\frac{d^2x}{dt^2} - \mu(1 - x^2)\frac{dx}{dt} + x = 0$$

dove:

- $x(t)$ è la variabile dipendente che rappresenta la posizione (o una generica grandezza fisica oscillante) al tempo t .
- μ è un parametro scalare non negativo che controlla la non linearità e lo smorzamento del sistema.

Il termine $-\mu(1 - x^2)\dot{x}$ agisce come un meccanismo di **smorzamento non lineare**. La sua analisi rivela il comportamento unico del sistema:

- Quando $|x| < 1$, il coefficiente $(1 - x^2)$ è positivo, e il termine di smorzamento è negativo ($-\mu(\dots)$). Questo corrisponde a un'**immissione di energia** nel sistema, che tende ad amplificare le oscillazioni e ad allontanare le traiettorie dall'origine $(0, 0)$.

- Quando $|x| > 1$, il coefficiente $(1 - x^2)$ è negativo, e lo smorzamento diventa positivo. Questo corrisponde a una **dissipazione di energia** che smorza le oscillazioni e riporta le traiettorie verso l'origine.

Questa dualità porta alla formazione di un **ciclo limite** indipendentemente dalla condizione iniziale (ad eccezione del punto di equilibrio instabile all'origine), la soluzione converge verso un'unica oscillazione periodica stabile nello spazio delle fasi.

Il parametro μ è cruciale per la difficoltà numerica del problema:

- Per $\mu = 0$, l'equazione si riduce a $\ddot{x} + x = 0$, l'oscillatore armonico semplice.
- Per valori piccoli di μ (es. $0 < \mu < 1$), il sistema è debolmente non lineare.
- Per valori grandi di μ (es. $\mu \gg 1$), il sistema diventa estremamente **stiff**. Le soluzioni, note come **oscillazioni di rilassamento** sono caratterizzate da lunghi periodi di lenta evoluzione seguiti da transizioni quasi istantanee. È proprio in questo regime che i solutori espliciti tradizionali falliscono o diventano computazionalmente proibitivi.

3.1.2 Conversion to First Order System

Per essere risolta dalla maggior parte dei solutori numerici standard e per essere formulata in modo agevole per le PINN, l'ODE del secondo ordine viene tipicamente convertita in un sistema di due ODE del primo ordine. Introduciamo un vettore di stato $\mathbf{y}(t) \in \mathbb{R}^2$ definito come:

$$\mathbf{y}(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \end{bmatrix} = \begin{bmatrix} x(t) \\ \frac{dx}{dt}(t) \end{bmatrix}$$

La derivata del vettore di stato rispetto al tempo è quindi:

$$\frac{d\mathbf{y}}{dt} = \begin{bmatrix} \dot{y}_1 \\ \dot{y}_2 \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \ddot{x} \end{bmatrix}$$

Sostituendo le definizioni, otteniamo le due equazioni del sistema:

- Dalla definizione di y_2 , abbiamo:

$$\dot{y}_1 = y_2$$

- Dall'equazione originale di Van der Pol, possiamo isolare $\ddot{x}$:

$$\ddot{x} = \mu(1 - x^2)\dot{x} - x$$

Sostituendo $x = y_1$ e $\dot{x} = y_2$, otteniamo:

$$\dot{y}_2 = \mu(1 - y_1^2)y_2 - y_1$$

Il problema test è quindi definito dal seguente sistema di ODE del primo ordine:

$$\frac{dy}{dt} = \mathbf{f}(t, \mathbf{y}) = \begin{bmatrix} y_2 \\ \mu(1 - y_1^2)y_2 - y_1 \end{bmatrix}$$

Questo sistema rappresenta la formulazione matematica che verrà fornita in input a tutti i solutori numerici e alla PINN per gli esperimenti di questo studio.

3.2 Numerical Methods Implementations

Per garantire un confronto rigoroso e riproducibile, tutti i solutori numerici classici sono stati implementati come funzioni `.m` dedicate all'interno dell'ambiente MATLAB. Questo approccio assicura il pieno controllo su ogni aspetto dell'algoritmo, dalla gestione del passo di integrazione alla risoluzione delle non linearità. Tutte le implementazioni sono disponibili e consultabili nel repository del progetto, in particolare all'interno della directory `src/ode`.

Il cuore di ogni simulazione è la funzione `van_der_pol.m`, che definisce il sistema di ODE da risolvere. Questa funzione accetta il tempo `t`, il vettore di stato `y` e il parametro `mu`, restituendo la derivata del vettore di stato `dydt`. L'utilizzo di un **function handle** (`@van_der_pol`) permette di passare questa definizione in modo flessibile a ciascun solutore, assicurando che tutti operino sullo stesso identico problema matematico.

N.B: Sebbene l'oscillatore di Van der Pol sia stato utilizzato in questa sezione come esempio primario per illustrare nel dettaglio le tecniche di implementazione dei solutori, si specifica che l'analisi di benchmark finale, presentata nei capitoli successivi, è stata condotta su **tutti i sistemi fisici** descritti teoricamente nella Sezione 2.1. Le rispettive funzioni MATLAB sono reperibili nella directory `src/systems` del repository.

3.2.1 Explicit Method Implementations (Euler Explicit, Leapfrog, Runge-Kutta)

I metodi espliciti, essendo diretti nel calcolo, sono stati implementati tramite un ciclo `for` che itera su un vettore temporale pre-calcolato. Questo vettore viene generato utilizzando la funzione `linspace(t_start, t_end, num_steps)`, che impone un **passo di integrazione fisso** h .

- **Eulero Esplicito** (`explicit_euler.m`): All'interno del ciclo, la soluzione al passo $n + 1$ viene calcolata direttamente dalla formula vettoriale:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \cdot \mathbf{f}(t_n, \mathbf{y}_n)$$

L'implementazione è la più semplice e richiede una sola valutazione del function handle `@van_der_pol` per ogni passo.

- **Leapfrog** (`leapfrog.m`): Essendo un metodo a due passi, la sua implementazione richiede un'inizializzazione specifica. Come visibile nel codice, il primo passo (y_1) viene calcolato con il metodo di Eulero Esplicito. Le iterazioni successive del ciclo `for` applicano la formula standard:

$$\mathbf{y}_{n+1} = \mathbf{y}_{n-1} + 2h \cdot \mathbf{f}(t_n, \mathbf{y}_n)$$

- **Runge-Kutta di 4° Ordine** (`runge_kutta.m`): Questa funzione implementa il classico schema RK4. Per ogni passo temporale, vengono calcolate le quattro pendenze intermedie $\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3, \mathbf{k}_4$ attraverso quattro chiamate al function handle `@van_der_pol`. La soluzione viene quindi aggiornata con la loro media pesata, come da formula.

3.2.2 Implicit Method Implementations (Euler Implicit, Crank-Nicolson)

I metodi impliciti richiedono un approccio particolare, poiché l'incognita $\mathbf{y}_{n+1}$ non può essere isolata algebricamente. Ad ogni passo temporale, è necessario risolvere un sistema di equazioni non lineari.

- **Formulazione del Problema di Root-Finding**: L'equazione implicita viene riformulata come un problema di ricerca di radice. Ad esempio, per il **metodo di Crank-Nicolson**, la funzione residuo $\mathbf{G}(\mathbf{y}_{n+1})$ da annullare è:

$$\mathbf{G}(\mathbf{y}_{n+1}) = \mathbf{y}_{n+1} - \mathbf{y}_n - \frac{h}{2} [\mathbf{f}(t_n, \mathbf{y}_n) + \mathbf{f}(t_{n+1}, \mathbf{y}_{n+1})] = \mathbf{0}$$

- **Soluzione Numerica** (`implicit_euler.m`, `crank_nicolson.m`): Invece di utilizzare un solutore generico, l'implementazione risolve il sistema non lineare $\mathbf{G}(\mathbf{z}) = \mathbf{0}$ ad ogni passo temporale tramite un **metodo iterativo di Newton-Raphson**. Questo approccio è più effi-

ciente e offre un maggiore controllo. L'iterazione di Newton per trovare la radice di $\mathbf{G}(\mathbf{z})$ è data da: $\mathbf{z}_{k+1} = \mathbf{z}_k - [J_G(\mathbf{z}_k)]^{-1} \mathbf{G}(\mathbf{z}_k)$ dove $\mathbf{z}_k$ è l'approssimazione alla k -esima iterazione e J_G è la **matrice Jacobiana** del residuo $\mathbf{G}$ rispetto a $\mathbf{z}$.

```

1  function Y = euler_explicit(f, t, y0)
2
3      h = t(2) - t(1);
4      N = numel(t);
5      D = numel(y0);
6      Y = zeros(D, N);
7      Y(:,1) = y0;
8
9      for n = 1:N-1
10         Y(:,n+1) = Y(:,n) + h * f(t(n), Y(:,n));
11     end
12 end

```

```

1  function Y = euler_implicit(f, t, y0, tol, maxit)
2
3      if nargin < 4 || isempty(tol), tol = 1e-8; end
4      if nargin < 5 || isempty(maxit), maxit = 20; end
5
6      N = numel(t);
7      D = numel(y0);
8      Y = zeros(D, N);
9      Y(:,1) = y0;
10
11     for n = 1:N-1
12         h = t(n+1) - t(n);
13         tn1 = t(n+1);
14
15         y = Y(:,n) + h * f(t(n), Y(:,n));
16
17         for it = 1:maxit
18             F = y - Y(:,n) - h * f(tn1, y);
19             if norm(F,2) < tol, break; end
20             J = jacobian(@(z) z - Y(:,n) - h * f(tn1, z), y);
21             dy = -J \ F;
22             y = y + dy;
23             if norm(dy,2) < tol, break; end
24         end
25         Y(:,n+1) = y;
26     end
27 end

```

```

1  function Y = crank_nicolson(f, t, y0, tol, maxit)
2
3      if nargin < 4 || isempty(tol), tol = 1e-8; end
4      if nargin < 5 || isempty(maxit), maxit = 20; end
5
6      N = numel(t);
7      D = numel(y0);
8      Y = zeros(D, N);
9      Y(:,1) = y0;
10
11     for n = 1:N-1
12         h = t(n+1) - t(n);
13         tn = t(n); tn1 = t(n+1);
14         fn = f(tn, Y(:,n));
15
16         y = Y(:,n) + h * fn;
17
18         for it = 1:maxit
19             F = y - Y(:,n) - (h/2) * ( fn + f(tn1, y) );
20             if norm(F,2) < tol, break; end
21             J = eye(D) - (h/2) * jacobian(@(z) f(tn1, z), y);
22             dy = -J \ F;
23             y = y + dy;
24             if norm(dy,2) < tol, break; end
25         end
26         Y(:,n+1) = y;
27     end
28 end

```

```

1  function Y = leapfrog(accel, t, y0)
2
3      N = numel(t); D = numel(y0); d = D/2;
4
5      if mod(D,2) ~= 0
6          error('leapfrog expects an even-sized state y=[x; v].');
7      end
8
9      Y = zeros(D, N);
10     x = y0(1:d,1);
11     v = y0(d+1:end,1);
12     Y(:,1) = y0;
13
14     for n = 1:N-1
15         h = t(n+1) - t(n);
16         a_n = accel(t(n), x, v);
17         v_half = v + 0.5*h * a_n;
18         x_new = x + h * v_half;
19         a_np1 = accel(t(n+1), x_new, v_half);
20         v_new = v_half + 0.5*h * a_np1;
21         x = x_new; v = v_new;
22         Y(:,n+1) = [x; v];
23     end
24 end

```

```

1  function Y = runge_kutta(f, t, y0)
2
3      N = numel(t);
4      D = numel(y0);
5      Y = zeros(D, N);
6      Y(:,1) = y0;
7
8      for n = 1:N-1
9          h = t(n+1) - t(n);
10         tn = t(n); yn = Y(:,n);
11         k1 = f(tn, yn);
12         k2 = f(tn + h/2, yn + h/2 * k1);
13         k3 = f(tn + h/2, yn + h/2 * k2);
14         k4 = f(tn + h, yn + h * k3);
15         Y(:,n+1) = yn + (h/6) * (k1 + 2*k2 + 2*k3 + k4);
16     end
17 end

```

3.3 PINN Implementation

L'implementazione della PINN è stata realizzata interamente in ambiente MATLAB, sfruttando le funzionalità avanzate del **Deep Learning Toolbox**. Questo toolbox fornisce gli strumenti essenziali per la definizione di architetture di rete personalizzate, il calcolo dei gradienti tramite differenziazione automatica e l'ottimizzazione dei parametri. L'approccio adottato trasforma la risoluzione dell'ODE in un problema di ottimizzazione non vincolata, il cui codice è reperibile nella directory `src/PINN/`.

3.3.1 Neural Network Architecture

Il cuore della PINN è una rete neurale di tipo *feed-forward* o Multi-Layer Perceptron (MLP), che funge da approssimatore universale per la soluzione del sistema di ODE. La rete è stata progettata per mappare la variabile temporale di input allo stato del sistema in output.

- **Input Layer:** La rete accetta un singolo input scalare, corrispondente al tempo t .
- **Hidden Layers:** L'architettura prevede una serie di strati nascosti densamente connessi. Per gli esperimenti condotti, sono stati utilizzati **N strati nascosti, ciascuno composto da K neuroni**. Questi valori sono iperparametri chiave che vengono sintonizzati per bilanciare la capacità espressiva della rete e il rischio di overfitting.
- **Funzione di Attivazione:** Tra ogni strato nascosto è stata interposta una funzione di attivazione non lineare. La scelta è ricaduta

sull'**arcotangente iperbolica** ($\tanh$). Questa funzione è C-infinita e il suo output è limitato nell'intervallo $(-1, 1)$, proprietà che favoriscono la stabilità durante l'addestramento.

- **Output Layer:** Lo strato di output è un **fullyConnectedLayer** con un numero di neuroni pari alla dimensionalità del sistema di ODE. Per l'oscillatore di Van der Pol, l'output è un vettore di due elementi, $\mathbf{y}_{NN} = [y_1, y_2]^T$, che approssima lo stato $[x, \dot{x}]^T$. Non viene applicata alcuna funzione di attivazione sullo strato di output per permettere alla soluzione di assumere qualsiasi valore reale.

L'intera architettura è definita in MATLAB come workflow di `dlarray` per consentire cicli di addestramento personalizzati.

3.3.2 Cost Function and Sampling Points

La funzione di costo, implementata come una funzione MATLAB separata, è la somma pesata di due componenti: l'errore sui dati e il residuo dell'equazione differenziale.

$$L(\theta) = L_{data}(\theta) + \lambda L_{physics}(\theta)$$

- **Punti di Campionamento:**

1. **Punto Iniziale:** Per il termine L_{data} , viene utilizzato un singolo punto corrispondente alla condizione iniziale $(t_0, \mathbf{y}_0)$.
2. **Punti di Collocazione:** Per il termine $L_{physics}$, viene generato un insieme di N_c **punti di collocazione**. Come visibile nel codice `pinn_van_der_pol.m`, questi punti

sono campionati casualmente in modo uniforme all'interno dell'intervallo di integrazione $[t_{start}, t_{end}]$ utilizzando la funzione `rand`.

- **Calcolo del Residuo Fisico:** Il calcolo del termine $L_{physics}$ richiede la derivata prima dell'output della rete rispetto al suo input. Questa operazione critica è gestita dalla **differentiazione automatica** di MATLAB. Specificamente, si utilizza la funzione `dlgradient` per calcolare $\frac{dy_{NN}}{dt}$. Il residuo $\mathbf{r}(t, \theta)$ è quindi calcolato come:

$$\mathbf{r}(t, \theta) = \frac{d\mathbf{y}_{NN}(t, \theta)}{dt} - \mathbf{f}(t, \mathbf{y}_{NN}(t, \theta))$$

dove $\mathbf{f}$ è il function handle del sistema di Van der Pol.

- **Formulazione della Loss:** I due termini della loss sono calcolati come l'errore quadratico medio (MSE):

$$L_{data}(\theta) = \text{mse}(\mathbf{y}_{NN}(t_0, \theta) - \mathbf{y}_0)$$

$$L_{physics}(\theta) = \frac{1}{N_c} \sum_{j=1}^{N_c} \|\mathbf{r}(t_{c,j}, \theta)\|_2^2$$

dove la somma viene eseguita su tutti i punti di collocazione.

3.3.3 Optimization and Training

Il processo di addestramento minimizza la funzione di costo $L(\theta)$ per trovare i parametri ottimali θ^* della rete.

- **Ottimizzatore:** Sono stato scelti gli ottimizzatori **Adamn** (Adaptive Moment Estimation) e SGD sta (Stochastic Gradient Descent).
- **Learning Rate:** Il tasso di apprendimento è un iperparametro cruciale che determina l'ampiezza dei passi durante l'ottimizzazione. Per gli esperimenti è stato utilizzato un valore di $\eta = 1 \times 10^{-3}$, soggetto a possibile decadimento (*decay*) durante l'addestramento per affinare la convergenza.
- **Ciclo di Addestramento:** L'addestramento è implementato come un ciclo `for` che itera per un numero predefinito di epoche. Ad ogni epoca:
 1. Viene chiamata la funzione `dlfeval`, che valuta la funzione di costo e i suoi gradienti rispetto ai parametri della rete, $\nabla_{\theta} L(\theta)$.
 2. L'ottimizzatore Adam aggiorna i parametri θ della rete (pesi e bias) utilizzando i gradienti calcolati. La funzione `adamupdate` di MATLAB gestisce questo passo.

Il processo viene ripetuto fino al raggiungimento del numero massimo di epoche, momento in cui i parametri finali θ^* definiscono il modello PINN addestrato, pronto per la valutazione.

```

1  function net = model(cfg)
2
3      if nargin < 1 || isempty(cfg), cfg = struct(); end
4      cfg = fill_defaults(cfg, struct( ...
5          %% Configuration: input_dim, outpu_dim, hidden_sizes,
6              activation, dropout, dtype
7      ));
8      if cfg.dropout < 0 || cfg.dropout > 1
9          error('cfg.dropout must be in [0,1].');
10     end
11     if ~ismember(cfg.activation, {'tanh','relu','swish'})
12         error('cfg.activation must be one of: tanh | relu | swish.');

```

```

1  function [L, terms] = losses(net, sys, t0, y0, t_col, cfg, data)
2
3      if nargin < 6 || isempty(cfg), cfg = struct; end
4      if ~isfield(cfg, 'lambda_res'), cfg.lambda_res = 1.0; end
5      if ~isfield(cfg, 'lambda_ic'), cfg.lambda_ic = 1.0; end
6      if ~isfield(cfg, 'lambda_data'), cfg.lambda_data = 0.0; end
7
8      [R, ~, ~] = residual(net, sys, t_col, 'train');
9      L_res = mean(R.^2, 'all');
10
11     t0_d1 = dlarray(reshape(t0,1,[]));
12     Y0 = net.forward(t0_d1, net.params, 'eval');
13     L_ic = mean((Y0 - y0).^2, 'all');
14
15     if nargin >= 7 && ~isempty(data) && isfield(data, 't') && isfield(
16         data, 'y') && ~isempty(data.t)
17         Yd = net.forward(data.t, net.params, 'eval');
18         L_data = mean((Yd - data.y).^2, 'all');
19     else
20         L_data = L_res*0;
21     end
22
23     L = cfg.lambda_res * L_res + cfg.lambda_ic * L_ic + cfg.
24         lambda_data * L_data;
25
26     terms = struct('res', L_res, 'ic', L_ic, 'data', L_data);
27
28     end

```

```

1  function [R, dYdt, Y] = residual(net, sys, t, mode)
2
3      if nargin < 4, mode = 'eval'; end
4      if ~isa(t, 'dlarray'), t = dlarray(t); end
5
6      Y = net.forward(t, net.params, mode);
7
8      D = size(Y,1);
9      N = size(Y,2);
10     dYdt = dlarray(zeros(D, N, 'like', Y));
11
12     for i = 1:D
13         s = sum(Y(i,:), 'all');
14         gi = dlgradient(s, t, 'EnableHigherDerivatives', true);
15         dYdt(i,:) = gi;
16     end
17
18     fY = sys.f(t, Y);
19     R = dYdt - fY;
20 end

```

```

1  function t_col = collocation(tspan, N, mode, seed)
2
3      if nargin < 3 || isempty(mode), mode = 'random'; end
4      if nargin >= 4 && ~isempty(seed), rng(seed); end
5      t0 = tspan(1); t1 = tspan(2);
6
7      switch lower(mode)
8          case 'random'
9              t = t0 + (t1 - t0) * rand(1, N, 'single');
10             t(1) = t0;
11          case 'grid'
12             t = linspace(t0, t1, N);
13          otherwise
14             error('Unknown mode %s', mode);
15      end
16      t_col = dlarray(t);
17 end

```

```

1  function [tgrid, Ypred] = evaluate(net, tspan, N)
2
3      if nargin < 3, N = 1001; end
4
5      tgrid = linspace(tspan(1), tspan(2), N);
6      Y = net.forward(dlarray(tgrid), net.params, 'eval');
7      Ypred = gather(extractdata(Y));
8  end

```

```

1  function [net, hist] = train(net, sys, tspan, ic, cfg, data)
2
3      if nargin < 6, data = []; end
4      if nargin < 5 || isempty(cfg), cfg = struct(); end
5
6      cfg = fill_defaults(cfg, struct( ...
7          'epochs',      2000, ...
8          'batch_size',  128, ...
9          'collocation_N', 4096, ...
10         'seed',        42, ...
11         'lr',          1e-3, ...
12         'momentum',    0.9, ...
13         'grad_clip',   5.0, ...
14         'loss_weights', struct('lambda_res',1,'lambda_ic',1,'
            lambda_data',0), ...
15         'optimizer',   'sgd', ...
16         'beta1',       0.9, ...
17         'beta2',       0.999, ...
18         'eps',         1e-8, ...
19         'lr_decay',    [], ...
20         'decay_every', [], ...
21     ));
22
23     rng(cfg.seed);
24     t_pool = collocation(tspan, cfg.collocation_N, 'random');
25     Nc = size(t_pool,2);
26
27     vel = struct();
28     m    = struct(); v = struct();
29
30     fn = fieldnames(net.params);
31     for k = 1:numel(fn)
32         vel.(fn{k}) = zeros(size(net.params.(fn{k})), 'like', net.
            params.(fn{k}));
33         m.(fn{k})    = zeros(size(net.params.(fn{k})), 'like', net.
            params.(fn{k}));
34         v.(fn{k})    = zeros(size(net.params.(fn{k})), 'like', net.
            params.(fn{k}));
35     end
36
37     num_batches = ceil(Nc / cfg.batch_size);
38
39     hist.epoch = zeros(cfg.epochs,1);
40     hist.loss  = zeros(cfg.epochs,1);
41     hist.res   = zeros(cfg.epochs,1);
42     hist.ic    = zeros(cfg.epochs,1);
43     hist.data  = zeros(cfg.epochs,1);
44
45     t0 = ic.t0;
46     y0 = ic.y0;
47
48     beta1 = cfg.beta1; beta2 = cfg.beta2; eps = cfg.eps;
49

```

```

50     lr = cfg.lr;
51
52     for ep = 1:cfg.epochs
53
54         if ~isempty(cfg.lr_decay) && ~isempty(cfg.decay_every) && cfg.
            decay_every > 0
55             if ep > 1 && mod(ep-1, cfg.decay_every) == 0
56                 lr = lr * cfg.lr_decay;
57             end
58         end
59
60         idx = randperm(Nc);
61         ep_loss = 0; ep_res = 0; ep_ic = 0; ep_data = 0;
62
63         for b = 1:num_batches
64             j = idx( (b-1)*cfg.batch_size + 1 : min(b*cfg.batch_size,
                Nc) );
65             t_batch = t_pool(:, j);
66
67             [L, terms, grads] = dlfeval(@loss_and_grads_fn, ...
68                 net, sys, t0, y0, t_batch, cfg.loss_weights, data);
69
70             if cfg.grad_clip > 0
71                 gnorm_sq = 0;
72                 for k = 1:numel(fn)
73                     gnorm_sq = gnorm_sq + sum(grads.(fn{k}).^2, 'all');
74                 end
75                 gnorm = sqrt(extractdata(gnorm_sq));
76                 if gnorm > cfg.grad_clip
77                     scale = cfg.grad_clip / gnorm;
78                     for k = 1:numel(fn)
79                         grads.(fn{k}) = grads.(fn{k}) * scale;
80                     end
81                 end
82             end
83
84             if use_adam
85                 t_adam = ep;
86                 for k = 1:numel(fn)
87                     g = grads.(fn{k});
88                     m.(fn{k}) = beta1 * m.(fn{k}) + (1 - beta1) * g;
89                     v.(fn{k}) = beta2 * v.(fn{k}) + (1 - beta2) * (g
                        .^2);
90
91                     mhat = m.(fn{k}) ./ (1 - beta1^t_adam);
92                     vhat = v.(fn{k}) ./ (1 - beta2^t_adam);
93
94                     net.params.(fn{k}) = net.params.(fn{k}) - lr *
                        mhat ./ (sqrt(vhat) + eps);
95                 end
96             else
97                 for k = 1:numel(fn)
98                     vtmp = cfg.momentum * vel.(fn{k}) - lr * grads.(fn
                        {k});

```

```

99         vel.(fn{k}) = vtmp;
100         net.params.(fn{k}) = net.params.(fn{k}) + vtmp;
101     end
102 end
103
104     ep_loss = ep_loss + double(gather(extractdata(L)));
105     ep_res  = ep_res  + double(gather(extractdata(terms.res)));
106     ;
107     ep_ic   = ep_ic   + double(gather(extractdata(terms.ic)));
108     ep_data = ep_data + double(gather(extractdata(terms.data))
109                                );
110 end
111
112 hist.epoch(ep) = ep;
113 hist.loss(ep)  = ep_loss / num_batches;
114 hist.res(ep)   = ep_res  / num_batches;
115 hist.ic(ep)    = ep_ic   / num_batches;
116 hist.data(ep)  = ep_data / num_batches;
117
118 print_every = 50;
119
120 do_metrics = isstruct(data) && ~isempty(data) && ...
121             ~isempty(data.t) && ~isempty(data.y);
122             isfield(data,'t') && isfield(data,'y') && ...
123
124 if do_metrics && (mod(ep, print_every) == 0 || ep == 1 || ep
125 == cfg.epochs)
126     Yhat = net.forward(data.t, net.params, 'eval');
127     Yhat = double(extractdata(Yhat));
128     Yref = double(extractdata(data.y));
129     M = compute_metrics(Yhat, Yref);
130     hist.mse(ep)      = M.mse;
131     hist.rmse(ep)     = M.rmse;
132     hist.mae(ep)      = M.mae;
133     hist.r2(ep)       = M.r2;
134 end
135
136 if ep == 1 || mod(ep, print_every) == 0 || ep == cfg.epochs
137     if do_metrics
138         fprintf(['[%4d/%4d] lr=%.2e | loss=%.3e |' ...
139                 ' MSE=%.3e, RMSE=%.3e, MAE=%.3f, R2=%.3f\n'],
140                 ...
141                 ep, cfg.epochs, lr, hist.loss(ep), ...
142                 hist.mse(ep), hist.rmse(ep), hist.mae(ep),
143                 hist.r2(ep)); ep, hist.res(ep), hist.ic(ep)
144                 ), hist.data(ep));
145     end
146 end
147 end
148
149 %% Utils functions...

```

3.4 Evaluation Metrics

Per condurre un confronto quantitativo e oggettivo tra i diversi solutori, è indispensabile definire un insieme di metriche standard per la valutazione delle performance. Una semplice ispezione visiva delle soluzioni prodotte, infatti, non è sufficiente per catturare le sottili differenze in termini di accuratezza e robustezza. Pertanto, ogni soluzione numerica generata $\hat{y}(t)$ è stata confrontata con la soluzione di riferimento $y(t)$ (ottenuta dai dataset pre-elaborati) su un insieme discreto di N punti temporali. Le seguenti metriche sono state adottate per quantificare l'errore.

3.4.1 Mean Squared Error - MSE

L'MSE è una delle metriche più comuni per la valutazione dell'errore di un modello. Misura la media delle differenze al quadrato tra i valori predetti e i valori reali.

Dati i valori reali y_i e i valori predetti $\hat{y}_i$ per $i = 1, \dots, N$:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

L'utilizzo del quadrato ha due effetti principali: primo, garantisce che gli errori siano tutti positivi, evitando che errori positivi e negativi si annullino a vicenda; secondo, **penalizza maggiormente gli errori di grande entità** rispetto a quelli piccoli. Questo rende l'MSE particolarmente sensibile ai valori anomali (*outliers*) e alle grandi deviazioni.

3.4.2 Normalized Mean Squared Error - NMSE

Per superare il problema della dipendenza dalla scala dei dati dell'MSE, si utilizza la sua versione normalizzata, l'NMSE. Questa metrica rapporta l'errore del modello alla varianza dei dati reali, fornendo una misura relativa della performance.

$$NMSE = \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

dove $\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$ è la media dei valori reali.

L'NMSE è una metrica adimensionale che permette di confrontare le performance di modelli su dataset con scale diverse. La sua interpretazione è molto potente:

- $NMSE = 0$: Corrispondenza perfetta.
- $NMSE < 1$: Il modello è più accurato di un semplice modello base che predice costantemente la media dei dati.

- $NMSE = 1$: Il modello ha la stessa performance del modello base (media).
- $NMSE > 1$: Il modello è *peggio* del predire semplicemente la media, indicando una performance molto scarsa.

3.4.3 Mean Absolute Error - MAE

Il MAE misura la media delle differenze in valore assoluto tra i valori predetti e quelli reali.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

A differenza dell'MSE, il MAE assegna lo stesso peso a tutti gli errori, indipendentemente dalla loro magnitudine. Questo lo rende **più robusto agli outliers** e più facilmente interpretabile.

3.4.4 Coefficient of Determination - R²

Il coefficiente di determinazione, noto come R^2 (o R-quadro), non misura l'errore in termini assoluti, ma la **proporzione della varianza** nei dati reali che è spiegata dal modello.

L' R^2 è definito a partire dalla scomposizione della varianza totale. La somma totale dei quadrati (SST), proporzionale alla varianza dei dati, è data da $SST = \sum_{i=1}^N (y_i - \bar{y})^2$. La somma dei quadrati dei residui (SSR), che rappresenta l'errore non spiegato dal modello, è $SSR = \sum_{i=1}^N (y_i - \hat{y}_i)^2$. L' R^2 è definito come:

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

Si noti che l' R^2 è direttamente correlato all'NMSE: $R^2 = 1 - NMSE$. La sua interpretazione è la seguente:

- $R^2 = 1$: Il modello spiega il 100% della variabilità dei dati; la corrispondenza è perfetta.
- $R^2 = 0$: Il modello non spiega affatto la variabilità e ha la stessa performance di un modello che predice la media.
- $R^2 < 0$: Il modello è peggiore del predire la media.

L'uso congiunto di queste quattro metriche fornisce una visione completa e robusta della performance di ciascun solutore, permettendo di valutarne non solo l'errore medio (MAE), ma anche la sensibilità agli errori grandi (MSE) e la capacità complessiva di "spiegare" la dinamica del sistema ($R^2/NMSE$).

4 Results

4.1 Dataset and Preprocessing

La validità di qualsiasi studio di benchmark numerico dipende in modo critico dalla qualità e dalla pertinenza dei dati utilizzati come riferimento. Per questo lavoro, si è scelto di non basarsi su soluzioni generate sinteticamente, ma di attingere a dataset pubblici e consolidati, rappresentativi di una varietà di sistemi fisici complessi. Questa scelta garantisce che il confronto avvenga su traiettorie realistiche e non banali.

Sono stati innanzitutto acquisiti differenti dataset dai seguenti repository pubblici sulla piattaforma Kaggle, una risorsa di riferimento per la comunità scientifica e del machine learning:

- **Three-Body Problem (TBP) Dataset:** <http://www.kaggle.com/datasets/sunyuanyi/tbp-dataset/tbp-dataset>
- **Rössler Chaotic Attractor:** <https://www.kaggle.com/datasets/henrychibueze/rossler-chaotic-attractor/rossler-chaotic-attractor>
- **Physics Attractor Time Series:** <https://www.kaggle.com/datasets/nikitricky/physics-attractor-time-series/physics-attractor-time-series>
- **Dynamics of Nonlinear Physical Systems Dataset:** <https://www.kaggle.com/datasets/ziya07/dynamics-of-nonlinear-physical-systems-dataset/dynamics-of-nonlinear-physical-systems-dataset>
- **Lorenz Attractor Official Dataset (CTF4Science):** <https://www.kaggle.com/datasets/dynamics-ai/ctf4science-lorenz-official-ds>

Ciascuno di questi dataset contiene serie storiche che descrivono l'evoluzione di sistemi dinamici noti per il loro comportamento non lineare o caotico. Tuttavia, i dati grezzi spesso includono informazioni aggiuntive o presentano una struttura non immediatamente compatibile con la formulazione dei problemi Test.

Pertanto, è stata condotta una fase di **pre-elaborazione** tramite script personalizzati sviluppati in Python. A seconda del tipo di sistema che si voleva analizzare (es. Lorenz, Rössler, Tre Corpi), sono state eseguite le seguenti operazioni:

1. **Caricamento del Dataset:** I dati sono stati caricati in un ambiente di analisi (es. tramite la libreria 'pandas').
2. **Selezione delle Colonne:** È stata operata una selezione delle colonne opportune per l'esperimento. Ad esempio, da un dataset contenente molteplici variabili, sono state estratte unicamente le colonne corrispondenti alle coordinate dello stato del sistema (es. x, y, z) e la colonna temporale.
3. **Adattamento alla Dimensione del Sistema:** Questa selezione ha permesso di conformare la struttura dei dati alla dimensionalità specifica del sistema di ODE in esame. Ad esempio, per l'attrattore di Lorenz, è stato generato un file contenente quattro colonne: $t, x(t), y(t), z(t)$.

Questo processo di cura e pulizia ha garantito che ogni solutore numerico e la rete PINN fossero Testati e validati rispetto a un set di dati di riferimento identico e coerente, assicurando così l'equità e la robustezza del confronto sperimentale.

4.2 Sperimental Setups

Per garantire la trasparenza e la riproducibilità dei risultati presentati, questa sezione descrive in dettaglio la configurazione hardware e software utilizzata per condurre tutti gli esperimenti numerici. Tutti i Test, sia per i solutori classici che per la PINN, sono stati eseguiti sulla medesima macchina al fine di assicurare un confronto equo delle performance computazionali.

4.2.1 Hardware environment

Le simulazioni sono state eseguite su un personal computer portatile LENOVO (Modello Sistema: 82H8, SKU: IdeaPad 3 15ITL6) con le seguenti specifiche tecniche:

- **Processore:**  Core™ i5-1155G7 di 11^a generazione, operante a una frequenza base di 2.50 GHz (4 core fisici, 8 processori logici).
- **Memoria RAM:** 8.00 GB di memoria fisica installata.
- **Sistema Operativo:** Microsoft Windows 11 Home (Versione 10.0.26100 build 26100), sistema a 64 bit.

Questa configurazione rappresenta un tipico ambiente di calcolo desktop, non specializzato per il calcolo ad alte prestazioni, rendendo i risultati del benchmark rappresentativi di ciò che un ricercatore o studente può ottenere su hardware di uso comune.

4.2.2 Software environment

L'ambiente di sviluppo e Testing è stato standardizzato per garantire coerenza tra gli esperimenti:

- **Piattaforma Principale:** Tutti i solutori numerici e l'architettura PINN sono stati implementati ed eseguiti in **MATLAB**.
- **Toolbox Utilizzati:**
 - Per l'implementazione della rete neurale, la definizione dei cicli di addestramento personalizzati e l'utilizzo della differenziazione automatica, è stato impiegato il **Deep Learning Toolbox** di MATLAB.
 - Le funzionalità di base del calcolo scientifico e della visualizzazione sono state gestite tramite le funzioni native di MATLAB.

- **Pre-elaborazione Dati:** Come menzionato nella Sezione 4.1, la fase di pre-elaborazione dei dataset scaricati da Kaggle è stata eseguita tramite script **Python**, avvalendosi delle librerie **pandas** per la manipolazione dei dati e **numpy** per le operazioni numeriche.

4.3 ODEs vs PINNs Benchmark

Questa sezione presenta i risultati quantitativi centrali di questo studio, mettendo a diretto confronto i solutori numerici classici e l'architettura Physics-Informed Neural Network. La sperimentazione è stata condotta utilizzando i sistemi dinamici descritti nel Capitolo 2, con un'attenzione particolare alla modulazione dei parametri che ne influenzano la complessità e la rigidità (*stiffness*), come il parametro μ per l'oscillatore di Van der Pol.

Le tabelle che seguono riassumono le performance di ciascun metodo implementato, con le rispettive configurazioni sperimentali:

	Train				Test			
	MSE	RMSE	MAE	R2	MSE	RMSE	MAE	R2
ODE	1.427e+02	1.194e+01	9.482e+00	0.282	1.347e+02	1.161e+01	9.188e+00	0.247
PINN	7.262e+01	8.522e+00	6.895e+00	0.635	7.263e+01	8.522e+00	6.909e+00	0.594

Tabella 4.1 - Dataset Lorenz:

lr = 3e-3; **momentum** = 0.2; **grad_clip** = 5.0; **loss_weights** = res : 0, ic : 0.3, data : 0.7

	Train				Test			
	MSE	RMSE	MAE	R2	MSE	RMSE	MAE	R2
ODE	2.260e+00	1.503e+00	1.067e+00	-0.067	5.596e+00	2.366e+00	1.993e+00	-3.399
PINN	4.001e-01	6.326e-01	4.359e-01	0.811	4.674e+00	2.162e+00	1.784e+00	-2.674

Tabella 4.2 - Dataset Van Der Pol (generated):

lr = 3e-2; **momentum** = 0.9; **grad_clip** = 1.5; **loss_weights** = res : 0, ic : 0.125, data : 0.875

	Train				Test			
	MSE	RMSE	MAE	R2	MSE	RMSE	MAE	R2
ODE	1.261e+01	3.551e+00	2.683e+00	-12.040	5.326e+00	2.308e+00	1.724e+00	-3.183
PINN	8.670e-01	9.311e-01	7.321e-01	0.104	1.172e+00	1.083e+00	8.714e-01	0.079

Tabella 4.3 - Dataset Non-Linear:

lr = 2e-1; **momentum** = 0.9; **grad_clip** = 0.5; **loss_weights** = res : 0, ic : 0.6, data : 0.4

4.4 Code Availability

A supporto della trasparenza e della riproducibilità scientifica di questo lavoro, l'intero codice sorgente utilizzato per gli esperimenti, incluse le implementazioni di tutti i solutori e dell'architettura PINN, è stato reso pubblicamente disponibile sul repository GitHub del progetto al seguente indirizzo: https://github.com/A-rgonaut/LM-32_PINNs_vs_ODE_Solvers_Numerical_Experiments_Benchmark.

5 Conclusions

Questo capitolo finale sintetizza i risultati del confronto sperimentale tra i solutori di ODE tradizionali e le Physics-Informed Neural Networks. Verrà fornita un'interpretazione critica dei risultati emersi dal benchmark, discutendo le implicazioni pratiche per la scelta del metodo di soluzione più appropriato. Infine, verranno analizzati i punti di forza e i limiti del presente studio, delineando possibili direzioni per la ricerca futura.

5.1 Results interpretation

L'analisi comparativa condotta ha rivelato un quadro complesso, dove nessuna metodologia si è dimostrata universalmente superiore in ogni scenario. Tuttavia, è emerso un chiaro trend: **le PINN, nella maggior parte dei casi, hanno mostrato performance di accuratezza superiori**, specialmente nell'affrontare sistemi non lineari e *stiff* su lunghi intervalli temporali.

Nonostante questo, i risultati hanno evidenziato che **le PINN non sono perfette** e la loro efficacia è fortemente influenzata dalla natura dei dati. Un'osservazione cruciale riguarda la loro gestione delle dinamiche ad alta frequenza. Quando la traiettoria della soluzione esibisce oscillazioni rapide e complesse, la rete neurale, a causa del suo "bias spettrale" che la porta a favorire approssimazioni a bassa frequenza, fatica a catturare ogni picco e valle. In questi casi, la PINN tende a convergere verso una **curva "mediata", che approssima l'involuppo generale della soluzione ma ne smussa i dettagli ad alta frequenza**. Al contrario, quando i dati non presentano alte frequenze e la dinamica è più "liscia", le PINN, con un'opportuna configurazione degli iperparametri e un numero sufficiente di epoche di addestramento, sono in grado di adattarsi quasi perfettamente alla soluzione di riferimento.

Dall'altro lato, i **solutori di ODE classici hanno mostrato un comportamento dicotomico**. La loro performance è indissolubilmente legata alla correttezza del modello matematico fornito. Se il sistema di equazioni scelto non descrive accuratamente la fisica sottostante ai dati di riferimento, il solutore numerico, pur essendo matematicamente corretto, produrrà una soluzione completamente errata, con errori catastrofici. Viceversa, in scenari in cui il modello è corretto e il sistema non presenta una rigidità proibitiva, i metodi tradizionali (in particolare quelli di ordine superiore come RK4) hanno dimostrato di poter **risultare talvolta persino più accurati delle PINN** soprattutto nella cattura di dettagli fini della traiettoria.

5.2 Sperimental Implications

Le osservazioni precedenti hanno importanti implicazioni pratiche per ricercatori e ingegneri. La scelta tra un solutore classico e una PINN non dovrebbe essere basata su una presunta superiorità assoluta, ma sulle specifiche esigenze del problema.

- **I solutori ODE tradizionali rimangono lo strumento d'elezione per la simulazione ad alta fedeltà di una singola traiettoria quando si dispone di un modello matematico consolidato e affidabile.** Sono deterministici, il loro comportamento è ben compreso e offrono un controllo preciso sull'errore di integrazione. Sono ideali per la **verifica** di un modello.
- **Le PINN emergono come lo strumento superiore per problemi di *apprendimento* e di **modellazione surrogata**.** Una volta addestrata, una PINN fornisce una rappresentazione funzionale, continua e differenziabile della soluzione, che può essere interrogata istantaneamente in qualsiasi punto del dominio. Questo è un vantaggio decisivo in applicazioni che richiedono valutazioni ripetute, come l'ottimizzazione, il controllo o l'analisi di sensitività parametrica. La loro natura "mesh-free" e la capacità di integrare dati sparsi le rendono inoltre adatte a problemi in cui la conoscenza del sistema è incompleta.

In sintesi, il paradigma si sposta: si sceglie un solutore classico per "risolvere un'equazione", si sceglie una PINN per "apprendere una funzione-soluzione".

5.3 Strengths and Limitations

Questo studio, come ogni lavoro sperimentale, presenta punti di forza e limiti che è importante riconoscere.

5.3.1 Strengths

- **Confronto Diretto:** Il lavoro ha fornito un benchmark quantitativo su metriche sia di accuratezza che di efficienza, applicato a sistemi complessi di rilevanza pratica.
- **Implementazione Trasparente:** L'utilizzo di codice open-source e dataset pubblici garantisce la piena riproducibilità e verificabilità dei risultati.
- **Analisi Sfumata:** Invece di dichiarare un vincitore assoluto, lo studio ha evidenziato i contesti specifici in cui ciascun approccio eccelle, fornendo una guida pratica alla scelta.

5.3.2 Limits and Future Prospects

- **Architettura della PINN:** L'architettura utilizzata è una rete MLP standard. Ricerche future potrebbero esplorare architetture più avanzate (es. con feature di Fourier) specificamente progettate per superare il bias spettrale e migliorare l'apprendimento delle alte frequenze.
- **Ottimizzazione degli Iperparametri:** La performance della PINN è fortemente sensibile alla scelta di iperparametri come il learning rate, la profondità della rete e il peso del termine di loss fisica (λ). Sebbene sia stata condotta una ricerca per trovare valori efficaci, un'esplorazione più esaustiva potrebbe portare a performance ancora migliori.
- **Generalizzazione dei Sistemi:** Lo studio si è concentrato su una classe di ODE note per il loro comportamento caotico. Ulteriori ricerche sono necessarie per estendere il confronto a altre classi di problemi, come le equazioni differenziali alle derivate parziali (PDE) o i sistemi su larga scala.

In conclusione, questo lavoro conferma che le PINN rappresentano un'aggiunta potente e versatile alla cassetta degli attrezzi del calcolo scientifico, non come un sostituto universale, ma come un complemento fondamentale ai metodi classici, con punti di forza unici in contesti di apprendimento e modellazione surrogata.

6 Bibliografia

- [Gal+21] Evangelos Galaris et al. “Numerical Solution of Stiff ODEs with Physics-Informed RPNs”. In: (2021), p. 30. DOI: 10.48550/arXiv.2108.01584.
- [ND22] Pouyan Nasiria and Roozbeh Dargazany. “Reduced-PINN: An Integration-Based Physics-Informed Neural Networks for Stiff ODEs”. In: (2022), p. 16. DOI: 10.48550/arXiv.2208.12045.
- [Bat23] Hubert Baty. “Solving stiff ordinary differential equations using physics informed neural networks (PINNs): simple recipes to improve training of vanilla-PINNs”. In: (2023), p. 16. DOI: 10.48550/arXiv.2304.08289.
- [Flo+24] Mario De Florio et al. “Physics-Informed Neural Networks for 2nd order ODEs with sharp gradients”. In: *Journal of Computational and Applied Mathematics* (2024), p. 115396. DOI: 10.1016/j.cam.2023.115396.
- [Lu+24] Kai-liang Lu et al. “Characteristic Performance Study on Solving Oscillator ODEs via Soft-constrained Physics-informed Neural Network with Small Data”. In: (2024), p. 24. DOI: 10.48550/arXiv.2408.11077.
- [Ziy24] Wang Ziyi. “A numerical method to solve PDE through PINN based on ODENet”. In: *Applied and Computational Engineering* (2024), pp. 251–259. DOI: 10.54254/2755-2721/68/20241467.
- [AOE25] Farea A., Yli-Harja O., and F. Emmert-Streib. “Using Physics-Informed Neural Networks for Modeling Biological and Epidemiological Dynamical Systems”. In: *Mathematics* (2025), p. 23. DOI: 10.3390/math13101664.
- [Cha+25] Nagesh Chandra et al. “Taylor-Model Physics-Informed Neural Networks (PINNs) for Ordinary Differential Equations”. In: (2025), p. 22. DOI: 10.48550/arXiv.2507.03860.